ROS2 Spring Workshop

Lab 4: The Seeder Pipeline (Services, Clients & Composition)

Project Context: The Triathlon Seeder

In **Phase 1** of your Final Project, your tractor drives through a field, and at specific locations it must "plant" a seed. Dropping a marker that other code can later verify.
How does your controller tell the simulator "plant a seed HERE"? Not with a Topic, those are for continuous streams like GPS or velocity. You need a one-shot, confirmable transaction: "Do this specific thing. Tell me when it's done. Tell me if it failed."
That's a **Service**. Today you build the seeder that will drive your Phase 1 score, first as a one-shot script, and then as a node that runs alongside your tractor and plants autonomously while it works the field.

Objective

In previous labs, we used Topics to stream data like a radio broadcast. Topics are terrible for things that must happen exactly once. If you publish "spawn a seed here!" on a Topic, your seed might get dropped, duplicated, or lost in traffic.

In this lab, you use Services to perform specific, one-time transactions. Then, in Part 6, you compose three nodes into a real-looking autonomous seeder pipeline.

You will learn:

- **Service Clients:** How to write code that asks another node to do a job.
- **The Handshake:** Understanding the Request/Response cycle.
- **Asynchronous Calls:** How to order food without freezing your robot's brain.
- **Node Composition:** How to extend a system by adding new nodes, not modifying old ones.
- **Sim vs. Reality:** Why "the obvious way" works in simulation but breaks on real hardware.

Pre-Lab Concept Primer: Radio vs. SkipTheDishes

1. Topics (The Radio Broadcast)

Think of a Topic as a live radio station.

- **The Publisher (DJ):** They talk continuously. They don't care if you are listening.
- **The Data:** If you tune in late, you missed the song. It's gone forever.
- **Use Case:** High-speed data like GPS or wheel speeds. We don't care about the packet from 1 second ago; we only want the newest one.

2. Services (SkipTheDishes)

Think of a Service as using a food delivery app. This is a **transaction**.

- **The Request (Place Order):** You select your items and hit "Checkout" (you send specific data).
- **The Process (Tracking):** The restaurant has to accept the order and cook it. This takes time!
- **The Response (Delivery):** The driver arrives and hands you the food (success) OR the app says "Restaurant cancelled" (failure).
- **Use Case:** Spawning seeds, resetting the simulation, taking a photo, triggering the sprayer. You need to know if it actually happened!

Feature	Topic (Radio)	Service (SkipTheDishes)
Communication	One-way (fire & forget)	Two-way (request & response)
Timing	Continuous (e.g., 30 Hz)	Discrete (event-based)
Reliability	Low (dropped packets OK)	High (must confirm success)

Table 1: The fundamental difference in ROS communication patterns.

3. The "Async" Revolution

This is the hardest concept in this lab. When we send a request, we have to wait for the answer. **How** we wait matters.

The Delivery Analogy

Scenario A: Synchronous (Blocking), "The Landline Call"
You call the restaurant. They put you on hold. You sit there holding the phone to your ear for 10 minutes. You cannot do anything else (like drive or work) because you are glued to the phone waiting for them to pick up.
Robot effect: the robot stops driving while waiting for the calculation.
Scenario B: Asynchronous (Non-Blocking), "The App"
You tap "Order" on SkipTheDishes. The app gives you a **tracking screen** (a "Future" object). You put your phone in your pocket and go back to playing video games. You only check the door when the app pings you.
Robot effect: the robot keeps driving while the calculation happens in the background.

For the Web Devs: Promises vs. ROS Futures

If you have written modern JavaScript, you already know how ROS 2 Services work under the hood!
When you call an asynchronous service in ROS 2, it is exactly like fetching an API in a web application. You don't freeze the app waiting for the server; you get a "tracking number" (`Promise`) and attach a callback for when the data arrives.
The Translation Guide:

Concept	JavaScript	ROS 2 (Python)
The Tracking Number	`Promise`	`Future`
The Async Call	`fetch('/api/plant')`	`client.call_async(req)`
The Callback	`.then(result => {...})`	`future.add_done_callback(...)`
The Blocking Wait	`await fetch(...)`	`rclpy.spin_until_future_complete()`

The Big Difference (Event Loop vs. Networking):
A JavaScript Promise manages asynchronous tasks within the browser or Node.js single-threaded event loop. A ROS 2 Future manages communication *across entirely different programs and machines* over a network middleware.

Part 1: Package Setup

1. Navigate to the source folder:

```
cd /workspaces/agtech_ros2/src

```

2. Create a new package:

```
ros2 pkg create --build-type ament_python lab4_services
```

3. Verify the structure: ensure the folder `lab4_services` was created.

Part 2: Manual Service Calls

Before writing code, let's manually make calls. We need to inspect the "interface" to know what data the service expects.

1. Run the simulator:

```
ros2 run turtlesim turtlesim_node
```

2. Check the service type. We need to know what "form" to fill out to use `/spawn`.

```
ros2 service type /spawn
# Output: turtlesim/srv/Spawn
```

3. Inspect the form:

```
ros2 interface show turtlesim/srv/Spawn
```

Output interpretation:

- Above the dashes (`---`): the **Request** (x, y, theta, name).
- Below the dashes: the **Response** (name).

4. Call the service manually. Try to spawn a seed at $x = 2, y = 2$:

```
ros2 service call /spawn turtlesim/srv/Spawn \
"{x: 2.0, y: 2.0, theta: 0.0, name: 'seed_1'}"
```

Syntax Spotlight: Anatomy of a Service Call

This command is complex; let's break it down:

- `ros2 service call`: The verb (initiate a transaction).
- `/spawn`: The **service name** (the specific phone number we are dialing).
- `turtlesim/srv/Spawn`: The **service type** (the language we must speak).
- `"{x: 2.0, ...}"`: The **request data**. This is written in **YAML** (Yet Another Markup Language).
  - It acts like a Python dictionary.
  - **Critical:** You must use double quotes `"` around the whole block and a space after every colon `:`.

Part 3: The Seeder Script

Writing a Service Client is complex. To save time, the logic has been pre-written in the lab resources. Your job is to install it and analyze it.

1. **Copy the script into your package:**

```
cp /workspaces/agtech_ros2/lab_resources/lab4/spawn_seed.py \
   /workspaces/agtech_ros2/src/lab4_services/lab4_services/
```

2. **Open the file for analysis:**

```
code /workspaces/agtech_ros2/src/lab4_services/lab4_services/spawn_seed.py

```

Code Analysis: Read the code in your editor. Note the key components:

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from turtlesim.srv import Spawn  # IMPORT THE SERVICE TYPE

class Seeder(Node):
    def __init__(self):
        super().__init__('seeder')

        # 1. CREATE CLIENT: Open the App
        self.client = self.create_client(Spawn, '/spawn')

        # 2. CHECK FOR SERVER: Is the simulator running?
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('Waiting for turtlesim /spawn service...')

        # 3. PREPARE THE ORDER (reusable request object)
        self.req = Spawn.Request()

    def plant(self, x, y, name):
        # Fill out the order form
        self.req.x = float(x)
        self.req.y = float(y)
        self.req.theta = 0.0
        self.req.name = name

        # ASYNC CALL: Returns a Future (tracking number) instantly
        future = self.client.call_async(self.req)

        # WAIT FOR DELIVERY: Block until the simulator confirms
        rclpy.spin_until_future_complete(self, future)
        return future.result()

def main(args=None):
    rclpy.init(args=args)
    node = Seeder()

    # Plant a short row of 3 seeds across the middle of the field
    for i in range(1, 4):
        node.get_logger().info(f"Planting seed {i}...")
        try:
            response = node.plant(float(i*2), 5.0, f"seed_{i}")
            node.get_logger().info(f"Success! Created: {response.name}")
        except Exception as e:
            node.get_logger().error(f"Failed: {e}")

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Listing 1: spawn_seed.py (Reference Only)

Code Walkthrough: Chunk by Chunk

Let's break down exactly what this script is doing:

- **Lines 1–4 (The Shebang & Imports):** We start with the standard Linux shebang and our usual ROS 2 imports (`rclpy`, `Node`). However, notice the critical difference from Lab 3: instead of importing a *message* type like `Twist` or `Pose`, we import a *service* type: `from turtlesim.srv import Spawn`. Service types define both the request (what we send) and the response (what we get back), bundled into one object.
- **Lines 6–18 (The Constructor, Opening the App):** The `__init__` method sets up three things. First, `create_client(Spawn, '/spawn')` opens a connection to the `/spawn` service, like installing

a delivery app on your phone. Second, the `while not self.client.wait_for_service()` loop checks whether the simulator is actually running and accepting orders; if not, the node waits patiently and logs a message every second instead of crashing. Third, `self.req = Spawn.Request()` creates a blank order form that we can fill out and reuse each time we want to plant a seed.

- **Lines 20–30 (The Plant Method, Placing an Order):** The `plant()` function is the core transaction. Lines 22–25 fill in the order form with the coordinates and name we want. Line 28 calls `call_async()`, which is like tapping "Place Order" on SkipTheDishes: it sends the request and immediately hands us back a **Future** (a tracking number), so our code isn't frozen waiting. Line 31 then calls `spin_until_future_complete()`, which tells ROS to keep processing events until the response arrives. Once it does, we return the result. This two-step pattern (fire async, then wait) is safe here because we are calling from `main()`, not from inside a callback.

- **Lines 32–49 (The Engine, Running the Planting Mission):** The `main()` function initializes ROS and creates the node, then runs a simple `for` loop to plant three seeds spaced 2 m apart across the middle of the field. Notice the `try/except` block wrapping each `plant()` call: if a seed name already exists (because you ran the script twice), the service will raise an error. Instead of crashing the whole program, we log a warning and move on to the next seed. This defensive pattern becomes critical in your Final Project, where a single crash means a failed run. Finally, we clean up with `destroy_node()` and `rclpy.shutdown()`, just like in every previous lab.

Syntax Spotlight: The Client

- `create_client(Type, Name)`: Opens the app connection.
- `wait_for_service()`: Checks if the restaurant is accepting orders.
- `call_async(req)`: Hits the "Place Order" button. It returns a **Future** (tracking number) instantly, so your code doesn't freeze.

Critical Warning: Do NOT mix Lab 3 and Lab 4

The trap: In Lab 3, you learned to use **callbacks** (the doorbell) to react to sensors. You might be tempted to put this service call inside your callback. For example, "if I see a weed, spawn a marker."
Why this fails (deadlock):

- ROS 2 nodes run on a **single-threaded executor** by default. There is one "brain" processing one event at a time.
- `spin_until_future_complete()` works by telling the executor to keep spinning until the response arrives.
- But if you're already *inside* a callback, the executor is currently busy running *your callback*. It can't pick up the response while it's stuck waiting on you.
- **Result:** the brain waits for the response, the response can't be delivered until the brain is free, and the robot freezes forever.

Rule: Only call `spin_until_future_complete()` from the `main` loop. If you must call a service from inside a callback (you will, in Part 6), use `add_done_callback` instead. We'll show you exactly how.

Part 4: Registration & Build

1. **Edit `setup.py`.** Add the entry point so ROS can find your script.

```
entry_points={
    'console_scripts': [
        'spawn_seed = lab4_services.spawn_seed:main',
    ],
},

```

2. **Build:**

```
cd /workspaces/agtech_ros2
colcon build --symlink-install
source install/setup.bash

```

3. **Run it:**

```
ros2 run lab4_services spawn_seed

```

You should see three little turtles appear in a row across the middle of the simulator. Congrats!! You just used a service to modify the simulation world.

Summary: The ROS 2 Toolkit (Cheat Sheet)

By now, you have learned the "big three" tools for making robots think. Use this table to decide which one to reach for. **In Part 6 you will use all three at once** for the first time.

Tool	Analogy	Control Direction	When to use?
Timer *(Lab 2)*	**Heartbeat**	**Internal** (clock triggers code)	**Periodic tasks:** "Check battery every 1 s." "Calculate speed at 100 Hz."
Subscriber *(Lab 3)*	**Doorbell**	**Inbound** (world triggers code)	**Reactions:** "If GPS says $X > 9$, stop." "If camera sees red, stop."
Service Client *(Lab 4)*	**SkipTheDishes**	**Outbound** (code triggers world)	**Transactions:** "Plant a seed here." "Reset the simulation." "Calibrate the camera."

Part 5: Challenge, The Precision Seeder

AgTech Context: Real automated seeders don't plant randomly. The tractor drives across the field in a grid pattern, dropping seeds at precise intervals to optimize plant density and yield.

Your job is to write a new script, `seed_field.py`, that plants a **3 row × 4 column grid of wheat** at 2 m spacing (12 seeds total). Use the template code in the lab resource folder of the same name as the basis.

Your Requirements:

- Spawn exactly **12 seeds** in a 3×4 grid, spacing 2 m.
- Teleport `turtle1` to each grid coordinate before planting using the teleport service.
- Name each seed `wheat_{row}_{col}` so you can verify them later.
- Wrap each `plant()` call in a `try/except` block so re-running your script doesn't crash when names collide.

Verifying Your Field

Once your script finishes, you can count the seeds using the command line. Each turtle that spawns successfully registers 3 services behind the scenes.
Run this command:

```
ros2 service list | grep wheat | wc -l
```

If your count equals $12 \times 3 = 36$, your field is fully planted!

Part 6: The Composed Seeder System

Up to this point in the semester you have run *one* node at a time: a publisher, then a subscriber, then a service client. Real ROS systems don't work like that. A real autonomous tractor runs dozens of nodes simultaneously — one for the GPS driver, one for the camera, one for path planning, one for steering control, one for the seeder, one for logging — all communicating over topics and services. Today you build your first multi-node system.

Three terminals. Three nodes. One pipeline.

The Big Idea: Composition Without Modification

Look at the controller you wrote in Lab 3. It drives the tractor based on the geofence. Now imagine you want to add seed-planting. The *wrong* instinct is to open `safety_stop.py` and tack a service client onto it. The *right* instinct is to leave it alone and write a new node that observes the tractor from outside.

Why? In a real codebase, you wouldn't have permission to modify the driving controller. It would be safety-certified, written by another team, or just too critical to risk breaking. Instead, you write your seeder as a *separate node* that subscribes to the same `/turtle1/pose` topic and acts on what it hears.

The driver doesn't know your seeder exists. They communicate *only* through topics. **This is the entire philosophy of ROS**, and Part 6 is where you finally get to feel it.

Part 6.1: Meet the Field Driver

Your Lab 3 controller drove a single paperclip turn. The **field driver** provided in your lab resources is the same idea, leveled up: instead of one row, it snakes through the field for N rows, like a real tractor working a field.

Copy it into your package:

```
cp /workspaces/agtech_ros2/lab_resources/lab4/field_driver.py \
   /workspaces/agtech_ros2/src/lab4_services/lab4_services/
```

A Service Inside the Driver

Scroll down a bit further in `field_driver.py` and you'll see something interesting: the driver itself uses a **service client** to teleport the turtle to `(START_X, START_Y)` when the node starts up. By default, turtlesim spawns turtle1 in the middle of the field at `(5.5, 5.5)`, which means the snaking pattern would only cover the top half. Teleporting to a corner first solves that.

Notice the driver uses `rclpy.spin_until_future_complete()` for this call — the *blocking* pattern from `spawn_seed.py`, not the `add_done_callback` pattern we'll use in 6.2. That's safe here because the call happens inside `__init__`, before `rclpy.spin(node)` is ever invoked. There is no callback in flight to deadlock against. Same lesson, applied where it actually fits.

Try it. Add the new entry point to `setup.py`:

```
'field_driver = lab4_services.field_driver:main',
```

Then rebuild and run. Open turtlesim in one terminal, then run the driver:

```
ros2 run lab4_services field_driver
```

Part 6.2: The Auto-Seeder (Stage 1)

Now you write the planter. It needs to do three things at once:

- **Listen** to `/turtle1/pose` so it always knows where the tractor is (Lab 3 pattern).
- **Tick** every 3 seconds (Lab 2 pattern).
- **Plant** a seed at the latest known pose on each tick (Lab 4 pattern).

That's the entire ROS toolkit, in one node, talking to the field driver through a topic and to turtlesim through a service. Welcome to real ROS.

Copy the boilerplate:

```
cp /workspaces/agtech_ros2/lab_resources/lab4/auto_seeder.py \
   /workspaces/agtech_ros2/src/lab4_services/lab4_services/
```

Open the file. The structure is laid out for you. Every line that needs ROS-specific code is marked `# TODO`. Your job is to fill them in. Refer to your existing code:

- For the **subscriber**: see `safety_stop.py` from Lab 3.
- For the **service client**: see `spawn_seed.py` from earlier today.
- For the **timer**: see your Lab 2 heartbeat publisher.

Critical: The Async Pattern Inside a Callback

The `plant()` method in `spawn_seed.py` used `rclpy.spin_until_future_complete()` to wait for the response inline. That worked because we called it from `main()`.
You cannot do that here. Your `timer_callback` is itself running inside the executor, so calling `spin_until_future_complete` from inside it would deadlock the brain (see the Critical Warning back in Part 3).
Instead, fire the request and tell ROS what to do when the response arrives:

```
future = self.spawn_client.call_async(req)
future.add_done_callback(self.spawn_done)
```

The timer callback fires the request and returns immediately. When the response eventually arrives, ROS calls `spawn_done` automatically. No waiting, no deadlock. The boilerplate already has the `spawn_done` method written for you — you just need to attach it.

Wire it up. Once the TODOs are filled in:

1. Add the entry point to `setup.py`: `'auto_seeder = lab4_services.auto_seeder:main',`
2. Rebuild and re-source.
3. Open three terminals.

Terminal	Command
1	`ros2 run turtlesim turtlesim_node`
2	`ros2 run lab4_services field_driver`
3	`ros2 run lab4_services auto_seeder`

You should see the tractor start snaking through the field, and every 3 seconds a new seed should appear right on its trail. Pause for a second and appreciate what just happened: **three independent nodes are talking to each other.**

Inspect the System Live

With all three terminals running, open a fourth and ask ROS what's happening:

```
ros2 node list
ros2 topic list
ros2 topic info /turtle1/pose
```

You'll see your two nodes alongside `/turtlesim`, and you'll see `/turtle1/pose` has *two* subscribers: the field driver (using it for navigation) and the auto-seeder (using it for planting). Same data, two consumers, no extra work.

Heads Up: Seeds Bunching on the Turns

Notice the seeds bunch up on the arcs and stack on top of each other after the tractor stops. That's because the seeder fires every 3 seconds *no matter what*. It doesn't know whether the tractor is mid-turn, working a row, or parked. On a real seeder you'd gate planting on motion (only plant if the tractor has moved a meaningful distance). Fixing this is bonus challenge #3 in Part 6.3.

Part 6.3: The Keystroke Seeder (Stage 2)

Auto-planting on a fixed timer is fine, but a real seeder fires when something *decides* a seed is needed; a sensor sees a gap, an operator presses a button, a path-planner reaches a waypoint. Let's wire up the simplest version of that: plant on **spacebar press**.

Reading the keyboard cleanly inside a Python ROS node is fiddly (raw terminal modes, non-blocking reads), so we hand you the input device as a separate helper:

```
cp /workspaces/agtech_ros2/lab_resources/lab4/key_trigger.py \
   /workspaces/agtech_ros2/src/lab4_services/lab4_services/
```

Open `key_trigger.py`. It's tiny: it reads keystrokes and publishes an empty message (`std_msgs/Empty`) on `/plant_seed` every time you hit space. That's its whole job.

The Decoupling Lesson

Your seeder is about to subscribe to `/plant_seed`. It will not know or care *who* is publishing to that topic. That means the same seeder node would work if the trigger came from:

- This keyboard helper (what we're using)
- A CLI command: `ros2 topic pub --once /plant_seed std_msgs/msg/Empty {}`
- A button on a physical controller
- A footswitch on a real tractor
- Another autonomous node deciding when to plant

The input device is a separate concern from the action. On a real robot you will rip out and replace input devices constantly; swap a keyboard for a joystick, swap a joystick for a touchscreen, swap a touchscreen for an autonomous trigger. Topic-based decoupling means none of those changes touch your seeder. Worth pausing on.

Now write the keystroke seeder. It's almost identical to your auto-seeder. Duplicate it:

```
cp /workspaces/agtech_ros2/src/lab4_services/lab4_services/auto_seeder.py \
   /workspaces/agtech_ros2/src/lab4_services/lab4_services/key_seeder.py
```

Open `key_seeder.py`. You need five small edits. The structure stays the same, you're just swapping the timer for a topic subscriber.

1. **Add an import** at the top:

```
from std_msgs.msg import Empty
```

2. **Rename the node** in `super().__init__(...)`:

```
super().__init__('key_seeder')
```

3. **Delete the timer line** in `__init__`. You don't want time-based planting:

```
# DELETE this line:
self.timer_ = self.create_timer(3.0, self.timer_callback)
```

4. **Add a subscriber to /plant_seed** in `__init__` (where the timer used to be):

```
self.trigger_sub = self.create_subscription(
    Empty, '/plant_seed', self.plant_now, 10)
```

5. **Rename `timer_callback` to `plant_now`** and add a `msg` parameter. The body stays the same:

```
def plant_now(self, msg):
    # ... existing body of timer_callback ...
```

The pose subscriber, service client, and `spawn_done` callback all stay exactly the same.

Gotcha: Callbacks Always Take a Message Argument

Every subscriber callback in ROS receives the message as its first parameter, even when the message type is `Empty` and carries no data. If you forget and write `def plant_now(self):`, you'll get this cryptic error the moment the first message arrives:

```
TypeError: plant_now() takes 1 positional argument but 2 were given
```

The fix is just adding `msg` to the signature. You don't have to use it inside the function. The *event* of receiving the message is the whole signal.

Don't forget the entry point in `setup.py`:

```
'key_seeder  = lab4_services.key_seeder:main',
'key_trigger = lab4_services.key_trigger:main',
```

Rebuild. Now you need *four* terminals:

Terminal	Command
1	`ros2 run turtlesim turtlesim_node`
2	`ros2 run lab4_services field_driver`
3	`ros2 run lab4_services key_seeder`
4	`ros2 run lab4_services key_trigger`

Click into Terminal 4 so it has keyboard focus, then watch the tractor work the field. Every time you tap **SPACE**, a seed appears wherever the tractor is right at that instant. Press `q` in Terminal 4 to quit the trigger cleanly.

Bonus Challenges!

- **Add an undo key.** Use the `/kill` service (look it up with `ros2 service list`) and have a second key in `key_trigger.py` delete the most recent seed.
- **Plant a row, not a single seed.** On spacebar, plant 5 seeds spaced 0.5 m apart in a line behind the tractor.
- **Suppress duplicate plants.** If the tractor hasn't moved more than 0.3 m since the last plant, ignore the spacebar press and log a warning. Forces you to compare two consecutive poses.

Deliverables

Submit a PDF containing:

1. **Precision Seeder (Part 5):** A screenshot of turtlesim after your `seed_field.py` script has completed, showing all 12 seeds in a 3×4 grid pattern.
2. **Composed System (Part 6):** A screenshot of turtlesim showing the tractor actively working the field with seeds planted along its trail (from either your auto-seeder or your keystroke seeder).
3. **Sim-Reality Reflection (3–5 sentences):** Turtlesim lets us teleport and spawn seeds perfectly every time. Think about a real tractor planting real seeds in a dirt field. What physical realities, mechanical errors, or environmental factors is this 2D simulation missing? Name at least two things a real robotic seeder software stack would have to handle that turtlesim completely ignores.

What's Next: Lab 5

You've now built a publisher (Lab 2), a subscriber (Lab 3), a service client (Lab 4), and your first multi-node pipeline. The four core ROS communication patterns. All in simulation, all reading perfect `/turtle1/pose` data piped straight from turtlesim.
Real robots don't have that luxury. Their sensors lie in mathematically structured ways, and your code has to compensate.
In Lab 5 you'll build your tractor's first real sensor: a webcam + AprilTag detector that computes the 3D pose of a printed marker. You'll also confront **camera calibration** (turning a simple webcam into a measurement instrument) and the **transform tree** (how ROS tracks "where is X relative to Y?"). It's the first lab where your code talks to physical hardware.